

🏠 > SciPy API > Statistical functions (`scipy.stats`) > `scipy.stats....`

scipy.stats.ttest_rel

```
scipy.stats.ttest_rel(a, b, axis=0, nan_policy='propagate',  
alternative='two-sided', *, keepdims=False)
```

[\[source\]](#)

Calculate the t-test on TWO RELATED samples of scores, a and b.

This is a test for the null hypothesis that two related or repeated samples have identical average (expected) values.

Parameters:

a, b : *array_like*

The arrays must have the same shape.

axis : *int or None, default: 0*

If an int, the axis of the input along which to compute the statistic. The statistic of each axis-slice (e.g. row) of the input will appear in a corresponding element of the output. If `None`, the input will be raveled before computing the statistic.

nan_policy : *{'propagate', 'omit', 'raise'}*

Defines how to handle input NaNs.

- `propagate`: if a NaN is present in the axis slice (e.g. row) along which the statistic is computed, the corresponding entry of the output will be NaN.
- `omit`: NaNs will be omitted when performing the calculation. If insufficient data remains in the axis slice along which the statistic is computed, the corresponding entry of the output will be NaN.
- `raise`: if a NaN is present, a `ValueError` will be raised.

alternative : *{'two-sided', 'less', 'greater'}, optional*

Defines the alternative hypothesis. The following options are available (default is 'two-sided'):

- 'two-sided': the means of the distributions underlying the samples are unequal.

[Skip to main content](#)

- 'greater': the mean of the distribution underlying the first sample is greater than the mean of the distribution underlying the second sample.

! Added in version 1.6.0.

keepdims : *bool, default: False*

If this is set to True, the axes which are reduced are left in the result as dimensions with size one. With this option, the result will broadcast correctly against the input array.

Returns:

result : `TtestResult`

An object with the following attributes:

statistic : *float or array*

The t-statistic.

pvalue : *float or array*

The p-value associated with the given alternative.

df : *float or array*

The number of degrees of freedom used in calculation of the t-statistic; this is one less than the size of the sample (`a.shape[axis]`).

! Added in version 1.10.0.

The object also has the following method:

confidence_interval(confidence_level=0.95)

Computes a confidence interval around the difference in population means for the given confidence level. The confidence interval is returned in a `namedtuple` with fields *low* and *high*.

! Added in version 1.10.0.

Notes

[Skip to main content](#)

Examples for use are scores of the same set of student in different exams, or repeated sampling from the same units. The test measures whether the average score differs significantly across samples (e.g. exams). If we observe a large p-value, for example greater than 0.05 or 0.1 then we cannot reject the null hypothesis of identical average scores. If the p-value is smaller than the threshold, e.g. 1%, 5% or 10%, then we reject the null hypothesis of equal averages. Small p-values are associated with large t-statistics.

The t-statistic is calculated as `np.mean(a - b)/se`, where `se` is the standard error. Therefore, the t-statistic will be positive when the sample mean of `a - b` is greater than zero and



1.13.1 (stable) ▼



`np.ndarray` before the calculation is performed. In this case, the output will be a scalar or `np.ndarray` of appropriate shape rather than a 2D `np.matrix`. Similarly, while masked elements of masked arrays are ignored, the output will be a scalar or `np.ndarray` rather than a masked array with `mask=False`.

References

https://en.wikipedia.org/wiki/T-test#Dependent_t-test_for_paired_samples

Examples

Try it in your browser!

```
>>> import numpy as np
>>> from scipy import stats
>>> rng = np.random.default_rng()
```

```
>>> rvs1 = stats.norm.rvs(loc=5, scale=10, size=500, random_state=rng)
>>> rvs2 = (stats.norm.rvs(loc=5, scale=10, size=500, random_state=rng)
...         + stats.norm.rvs(scale=0.2, size=500, random_state=rng))
>>> stats.ttest_rel(rvs1, rvs2)
TtestResult(statistic=-0.4549717054410304, pvalue=0.6493274702088672, df=499)
>>> rvs3 = (stats.norm.rvs(loc=8, scale=10, size=500, random_state=rng)
...         + stats.norm.rvs(scale=0.2, size=500, random_state=rng))
>>> stats.ttest_rel(rvs1, rvs3)
TtestResult(statistic=-5.879467544540889, pvalue=7.540777129099917e-09, df=499)
```

[Skip to main content](#)

[< scipy.stats.power_divergence](#)[scipy.stats.wilcoxon >](#)

© Copyright 2008-2024, The SciPy community.

Created using [Sphinx](#) 7.3.7.

Built with the [PyData Sphinx Theme](#) 0.15.2.